

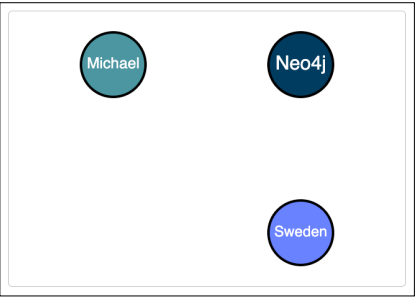
Core Components of Neo4j

1. Nodes

What is a Node?
A Node represents an entity or object in the real world.

Nodes are the vertices in a graph.
They represent objects or entities in your data model.
Imagine a social network, the entities (e.g. people, locations, companies) would be represented by nodes.

Example



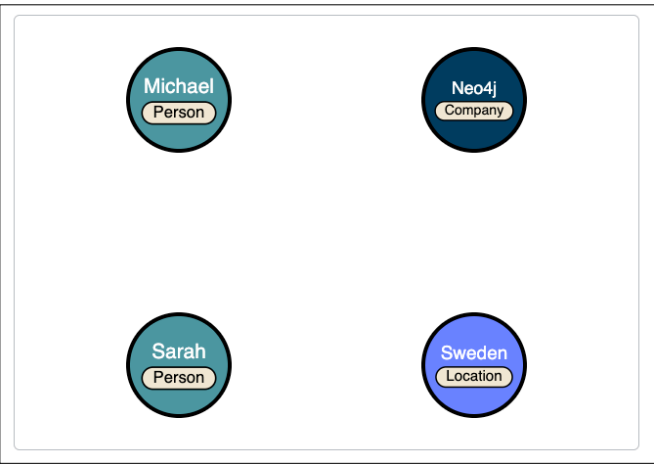
Here Nodes are
Michael
Sarah
Steve

2. Labels

What is a Label?
A Label is used to classify nodes.

Nodes are grouped by so categorized using labels.
Labels describe what the nodes are, for example, Person, Company, Location.

Example



Here Labels are
Person
Company
Location

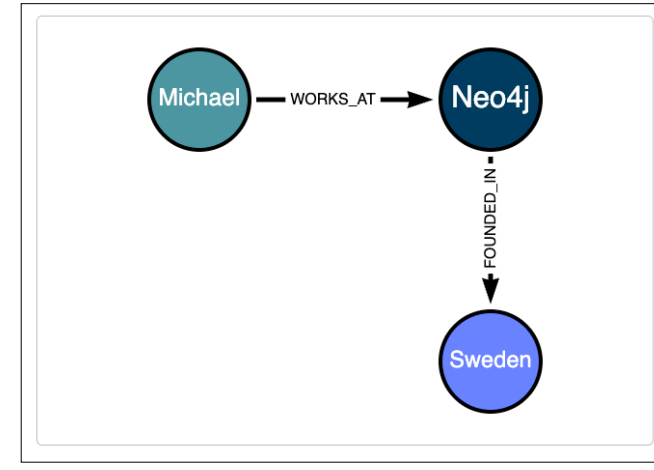
Nodes of the same type would have the same label.
Labels allow you to distinguish between different types of nodes and filter the graph.

3. Relationships

What is a Relationship?
A Relationship connects two nodes.

Relationships are the lines in the graph.
Relationships describe how nodes within the graph are connected to each other.

Example

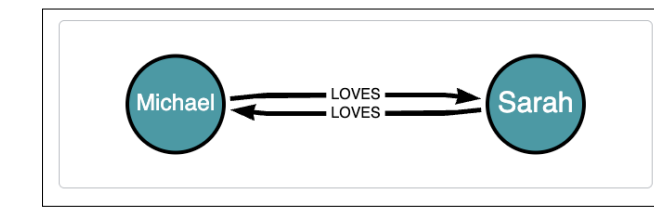


Here Relationships are
WORKS_AT
LOVES

A relationship in Neo4j connects two nodes, referred to as the start and end nodes.

Relationships have:
• type: WORKS_AT, LOVES, etc.
• direction: Michael WORKS_AT Sarah, Steve doesn't work at Michael.

Nodes can have multiple relationships to other nodes.
Multiple relationships can be used to describe bidirectional relationships.
In this example, two LOVES relationships are used to describe the status between Michael and Sarah.



Very common scenario that just because Michael loves Sarah, Sarah loves Michael back!

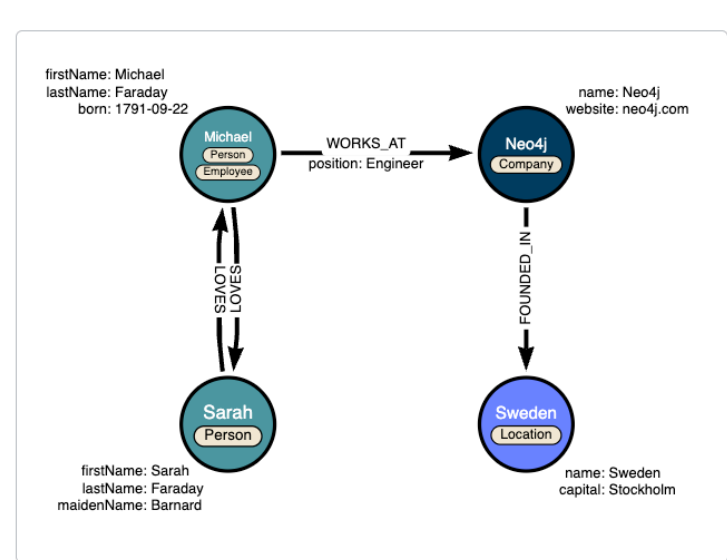
Use nodes for relationship types
You can use a relationship to represent:
• A personal connection: Person:MIKE, Person: Person:MAURICE, TO: PERSON
• A fact: Person:LOVE, IN: LOCATION, Person:ONE, GET: PERSON
• A hierarchy: Person:MAURICE, AT: OFFICE, Software:DEVELOPER, IN: COMPANY
• Any type of connection between 2 entities: Socially:CONNECTED, TO: PERSON

4. Properties

What are Properties?
Properties are key-value details stored on nodes or relationships.

You can store data against nodes and relationships as properties.
Properties are named key-value pairs, for example firstName, lastName, and position.

Example



Here Properties are
firstName, lastName, position, age, since

Nodes and relationships can have any number of properties, and those of the same type do not have to have the same properties (i.e. Neo4j is schemaless).
Properties have a type (string, boolean, string, int, etc) and can be unique identifier (string) for specific node labels.

Primitives

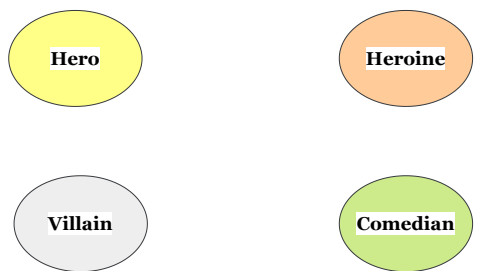
Simple Node Graph Example

Story

Hero loves Heroine.
Heroine also loves Hero.
Hero defeats Villain.
Villain has a friend who is a comedian.
Comedian respects Hero.
Villain is enemy of Hero.

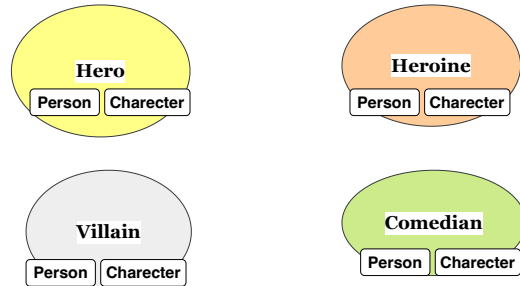
1. Nodes

Hero
Heroine
Villain
Comedian



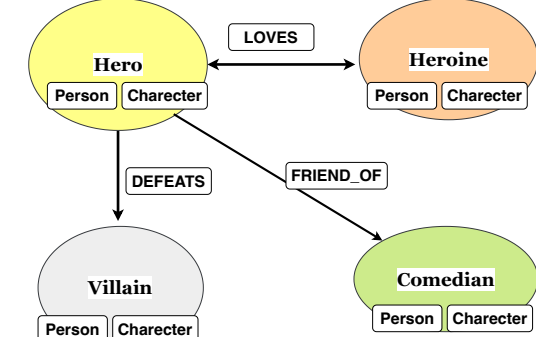
2. Labels

Hero has label: character, person
Heroine has label: character, person
Villain has label: character, person
Comedian has label: character, person



3. Relationships

Hero LOVES Heroine
Heroine LOVES Villain
Hero FRIENDS OF Comedian



4. Properties

Hero name = Ajax
Hero age = 30
Heroine name = Maria
Villain name = Bob
Comedian name = Bob

